The AMR Octree Dust Grid in MoCafe

Abstract

MoCafe's AMR mode (`par%grid_type = 'amr'`) replaces the uniform Cartesian density grid with an adaptive octree read from a generic AMR file (FITS / HDF5 / text) produced by the Python builders or by RAMSES / Illustris-TNG converters. The octree machinery is ported from LaRT but stripped to grey dust radiative transfer: each leaf carries a single dust opacity $\kappa(il)$ (extinction per unit code length), and the photon walks the tree via a precomputed face-neighbor table. Because dust scattering is leaf-independent, the scattering and peeling-off physics are shared verbatim with the Cartesian path; only the optical-depth integration (the raytrace) is octree-specific. This memo records the file schema, the octree build and traversal, the dust-opacity models and normalization, and the validation against a homogeneous sphere.

1. Model and file schema

The domain is a cube $[-L/2, L/2]^3$ (recentred on the origin) partitioned by an octree: cell index runs over internal and leaf cells, root = 1, octant = $1 + i_x + 2i_y + 4i_z$ with $i_x = 1$ if $x \geq$ the cell center. Only leaves carry opacity. The grid is read from a binary table (or text) with the mandatory columns

`x, y, z, level, nH, T, vx, vy, vz`

and the header keywords `NAXIS2` (= number of leaves), `BOXLEN`, and `ORIGINX/Y/Z`. Optional columns `metallicity`, `xHI`, `ndust` feed the dust models below. (`T` and the velocities are read for format compatibility but are ignored by the dust-only transport.) `amr_type = 'ramses'` is rejected with a pointer to the Python converter — MoCafe reads only the generic format.

2. Octree: build, neighbors, traversal

Build. `amr_build_tree` inserts each leaf from the root, creating internal nodes as needed, so the flat arrays of leaf positions and levels reconstruct the full tree (`parent`, `children(8,:)`, `level`, cell centers / half-widths, and the leaf↔cell maps). The tree arrays and the per-leaf opacity live in MPI-3 shared memory (one copy per node).

Neighbor table. `amr_build_neighbors` precomputes, for every cell, the same-level face neighbor in each of the six directions $(+x, -x, +y, -y, +z, -z)$ by querying the tree one cell-width past the face; ancestor returns (refinement gaps) are marked as "no neighbor." This makes each boundary crossing $O(1)$ instead of an $O(\log N)$ descent from the root.

Cell exit and next leaf. `amr_cell_exit` returns the distance to the nearest face of the current leaf and the face index. `amr_next_leaf` then follows the neighbor pointer and descends into the destination subtree using the two transverse sub-octant bits from the crossing point and the normal bit fixed *topologically* from the face index (so a photon sitting exactly on a face is routed without an ε nudge). For a space-filling octree this finds the entered leaf in $O(1)$; a return of 0 means the photon left the box (or hit a reflective / periodic boundary, handled per `par%xyz_symmetry` / `par%xy_periodic`).

3. Dust opacity and normalization

The per-leaf grey dust opacity is set, by `par%dust_density_law`, to one of

$$\texttt{global\_dgr}: \quad \kappa = n_{\rm H}\, C_{\rm ext}\, {\rm DGR}\, \ell_{\rm cm}, \tag{1}$$

$$\texttt{from\_file}: \quad \kappa = n_{\rm dust}\, C_{\rm ext}\, \ell_{\rm cm}, \tag{2}$$

$$\texttt{laursen09}: \quad \kappa = \frac{Z}{Z_{\rm ref}} \left(n_{\rm H} x_{\rm HI} + f_{\rm ion}\, n_{\rm H} (1 - x_{\rm HI}) \right) C_{\rm ext}\, \ell_{\rm cm}, \tag{3}$$

with $C_{\rm ext}$ = `par%cext_dust` and $\ell_{\rm cm}$ = `par%distance2cm`. The `laursen09` model requires an explicit `xHI` column (the $T \to x_{\rm HI}$ CIE path is deferred with temperature). When a system target is given, the whole population is rescaled by one factor so that the radial optical depth from the center to the $+z$ edge equals `par%taumax` (a pole traversal of the octree), or the volume-averaged $\kappa(L/2)$ equals `par%tauhomo`.

4. Reuse of the dust physics

AMR mode binds only the two raytrace procedure pointers (`raytrace_to_tau` → `raytrace_to_tau_amr`, `raytrace_to_edge` → `raytrace_to_edge_amr`); the forced first scattering, the Henyey–Greenstein (or Mueller-matrix) scattering, and the peeling-off are *identical* to the Cartesian dust path, because for grey dust they are leaf-independent. `raytrace_to_edge_amr` locates its start leaf from the photon position, so the peel works on a redirected copy of the photon without extra state. The photon's current leaf index (`photon%icell_amr`) is maintained across the forward walk and set at emission via `amr_find_leaf`.

5. Generators and converters (Python)

All generic AMR files share the schema of §2 and are produced by `python/AMR_grid/`:

- `AMR_grid.py` — dust-only octree builder (`AMRGrid`): uniform or density-gradient refinement, per-leaf `set_density` / `set_metallicity` / `set_neutral_fraction` / `set_dust_density`, and FITS/HDF5/text writers (the module-level `write_leaves` writes flat leaf arrays directly).
- `make_amr_sphere.py` — a compact analytic sphere (uniform level or radially refined) for validation.
- `convert_ramses_to_generic.py` — parses a native RAMSES output (`amr_*`/`hydro_*` Fortran-unformatted files, directly, no `yt`) and emits `x,y,z,level,nH` plus optional `metallicity` (metal hydro variable or uniform), `xHI` (ionization variable), and `ndust` (`--emit-ndust`).
- `convert_illustris_to_generic.py` — reads an Illustris-TNG gas cutout (local HDF5 or TNG.org API), converts comoving+h code units to physical kpc and $n_{\rm H}$, builds an octree (uniform or density-gradient, optionally matched to the local Voronoi cell size) via `AMRGrid`, and assigns `nH`, `metallicity` (`GFM_Metallicity`), `xHI` (`NeutralHydrogenAbundance`) by nearest-neighbor lookup, plus optional `ndust`.

Both converters are *dust only*: gas velocity and temperature (and the RAMSES energy/pressure and Illustris `InternalEnergy`/`Velocities`) are skipped. The Illustris converter was validated end-to-end on a real TNG50 cutout (uniform-level FITS and adaptive-level HDF5 grids both read by MoCafe and imaged); the RAMSES converter's generic-file output path is validated against MoCafe, while its Fortran-record reader is a faithful port of LaRT's. See `AMR_CONVERTERS_PLAN.md`.

6. Validation

A uniform-density sphere on a uniform-level-5 octree (32 cells across) was compared with the homogeneous Cartesian sphere at the same resolution (32^3), point source at the center, $\tau_{\rm max} = 1$, albedo 0.5, isotropic scattering, 2×10^6 photons:

section	Cartesian	AMR uniform	rel. diff.
Scattered	6.37856×10^1	6.37825×10^1	–0.005%
Direct	1.21791×10^2	1.21791×10^2	+0.000%
Direct0	3.31062×10^2	3.31062×10^2	+0.000%

The uniform octree reproduces the Cartesian result to Monte-Carlo precision. The *same* uniform sphere on a radially-refined octree (levels 3–6) agrees to –0.47% in the scattered total (Direct exact), the residual being the coarser cell sampling of the sphere limb; the agreement while crossing three refinement levels validates the octree's level-crossing ray traversal.

7. Scope

Gas velocity and temperature are deferred, so the LaRT Doppler width, Voigt profile, frequency tracking, per-cell velocity, diffuse Lyα emissivity, and the HEALPix inside-observer output are not ported. The pole-gap traversal helpers are kept for sparse (non-space-filling) octrees. See `AMR_CLUMPS_PLAN.md` Part A.